Project Proof Pack (UML + Output Evidence)

This document is designed for review, viva, examiner, and publication-proof submission.

1. Use Case Diagram

```mermaid
graph LR
Candidate((Candidate))
Recruiter((Recruiter))
Admin((Admin))

UC1[Register / Login]
UC2[Upload Resume]
UC3[View Recommendations]
UC4[Check ATS Eligibility]
UC5[Auto Apply Eligible Jobs]
UC6[Generate Skill Gap Roadmap]
UC7[Generate Adaptive Path]
UC8[Check Interview Readiness]

UC9[Create / Manage Jobs]
UC10[Review Applications]
UC11[Run Recruiter Copilot Ranking]
UC12[View Scam Risk]

UC13[View Platform Stats]
UC14[View Observability Summary]
UC15[View Research Evaluation]
UC16[Manage Users / Jobs]

Candidate --> UC1
Candidate --> UC2
Candidate --> UC3
Candidate --> UC4
Candidate --> UC5
Candidate --> UC6
Candidate --> UC7
Candidate --> UC8

Recruiter --> UC1
Recruiter --> UC9
Recruiter --> UC10
Recruiter --> UC11
Recruiter --> UC12

Admin --> UC1
Admin --> UC13
Admin --> UC14
Admin --> UC15
Admin --> UC16
```

2. Class Diagram (Logical)

```mermaid
classDiagram
class User {
+id: string
+fullName: string
+email: string
+role: Role
+isActive: boolean
}

class CandidateProfile {
+id: string
+skills: string[]
+experienceYears: int
+resumeUrl: string
+location: string
}

class RecruiterProfile {
+id: string
+designation: string
+companyId: string
}

class Company {
+id: string
+name: string
+industry: string
+location: string
}

class Job {
+id: string
+title: string
+skillsRequired: string[]
+experienceLevel: ExperienceLevel
+status: JobStatus
}

class Application {
+id: string
+status: ApplicationStatus
+appliedAt: datetime
+updatedAt: datetime
}

class Notification {
+id: string
+title: string
+message: string
+isRead: boolean
}

class RefreshToken {
+id: string
```

+tokenHash: string
+expiresAt: datetime
}

User "1" --> "0..1" CandidateProfile
User "1" --> "0..1" RecruiterProfile
Company "1" --> "0..*" RecruiterProfile
Company "1" --> "0..*" Job
RecruiterProfile "1" --> "0..*" Job
CandidateProfile "1" --> "0..*" Application
Job "1" --> "0..*" Application
User "1" --> "0..*" Notification
User "1" --> "0..*" RefreshToken
```

## 3. Candidate Application Sequence Diagram

```mermaid
sequenceDiagram
actor C as Candidate
participant W as Web App
participant API as Express API
participant DB as PostgreSQL/Prisma
participant Q as Notification Queue

C->>W: Login(email, password)
W->>API: POST /auth/login
API->>DB: Validate user credentials
DB-->>API: User record
API-->>W: accessToken + refreshToken

C->>W: Apply(jobId)
W->>API: POST /applications
API->>DB: Create application
DB-->>API: Application created
API->>Q: Enqueue recruiter notification
API-->>W: Application submitted

C->>W: Check ATS Score
W->>API: GET /applications/:id/ats-score
API->>DB: Fetch application + candidate + job
DB-->>API: Data
API-->>W: ATS score breakdown
```

4. Recruiter Copilot Activity Diagram

```mermaid
flowchart TD
A[Recruiter selects job] --> B[Fetch job applications]
B --> C[Parse candidate resume text]
C --> D[Extract candidate skills]
D --> E[Compute explainable score]
E --> F[Rank top candidates]
F --> G[Generate fairness snapshot]
```
```

```
G --> H[Return shortlist + explanation]
H --> I[Recruiter decision support]
```

5. Deployment Diagram

```mermaid
graph TB
U[User Browser]
FE[Vite React App :5173]
BE[Node Express API :4000]
DB[(PostgreSQL)]
R[(Redis optional)]
FS[(Resume Uploads FS)]

U --> FE
FE --> BE
BE --> DB
BE --> FS
BE --> R
```

6. ER Diagram (Submission-Friendly)

```mermaid
erDiagram
USER ||--o| CANDIDATE_PROFILE : has
USER ||--o| RECRUITER_PROFILE : has
COMPANY ||--o{ RECRUITER_PROFILE : employs
COMPANY ||--o{ JOB : posts
RECRUITER_PROFILE ||--o{ JOB : manages
CANDIDATE_PROFILE ||--o{ APPLICATION : submits
JOB ||--o{ APPLICATION : receives
USER ||--o{ NOTIFICATION : receives
USER ||--o{ REFRESH_TOKEN : owns

USER {
string id PK
string email
string role
bool isActive
}
CANDIDATE_PROFILE {
string id PK
string userId FK
int experienceYears
string resumeUrl
}
RECRUITER_PROFILE {
string id PK
string userId FK
string companyId FK
}
COMPANY {
```

```
string id PK
string name
}
JOB {
string id PK
string title
string recruiterId FK
string companyId FK
}
APPLICATION {
string id PK
string jobId FK
string candidateId FK
string status
}
```

7. Output Evidence Checklist (Screenshots)

Add these files under `docs/proof-assets/images/`.

#	Proof Item	Suggested File
1	Login page with role-ready credentials	`01-login-page.png`
2	Candidate dashboard after login	`02-candidate-dashboard.png`
3	Resume upload + analysis output	`03-resume-analysis.png`
4	Job recommendations with scores	`04-job-recommendations.png`
5	Eligibility report with ATS threshold	`05-eligibility-report.png`
6	Auto-apply confirmation modal	`06-auto-apply-confirmation.png`
7	Recruiter job creation section	`07-recruiter-create-job.png`
8	Recruiter copilot ranking output	`08-recruiter-copilot-ranking.png`
9	Job scam risk output	`09-scam-risk-report.png`
10	Admin monitoring panel	`10-admin-monitoring.png`
11	Admin research evaluation panel	`11-admin-research-evaluation.png`
12	API health proof (Postman/curl)	`12-api-health-postman.png`
13	API auth login response proof	`13-api-auth-login-postman.png`
14	API research evaluation response proof	`14-api-research-evaluation-postman.png`

8. Command Evidence (Optional but Strong)

Capture terminal outputs as screenshots for:

```bash
npm run build
npm run test
npx tsx scripts/eval.ts
curl -i http://localhost:4000/api/v1/health
```

9. PDF Export for Submission

You can include this proof pack in PDF by extending the generator:
- Source: `docs/PROJECT_PROOF_PACK.md`
- Existing generator script: `scripts/generate-doc-pdf.mjs`

If needed, create a second PDF from this markdown and attach both:

- `docs/FULL_PROJECT_DOCUMENTATION.pdf`
- `docs/PROJECT_PROOF_PACK.pdf`